

Assignment 3: End-to-End ML Versioning with Git, DVC & Google Drive

Samiha Hamid · Roll no. 23L-1018 · **Repository:** <https://github.com/Mia131103/fashion-ann-pipeline> · **DVC remote:** Google Drive folder [1ngK-GvQZmEW2jNMASRhxAoMcZwGj07z](#) · **Tags:** v1 = 7b71205, v2 = 1493de1 · **Final merge:** main @ 9d5d291

The ANN reaches **87.74 %** (v1) and **88.14 %** (v2) test accuracy, above the 85 % target. The pipeline runs end-to-end with one dvc repro, and every data/model version is on the Google Drive remote (sync check in E5).

Part A: Git fundamentals and advanced commands

A1-A2. First commit 6fee428 on main: README.md and a .gitignore for .venv, __pycache__/, *.py[oc] (later .env, /models; see C4). dev (git checkout -b dev) then received **21 commits**, about one per script or config: 7f5d237 prepare.py · dfb8989 preprocess.py + params.yaml · 73c5015 train.py · 1663628 evaluate.py · c5f22d0 dvc init · 0e53484 dvc add · 7b71205 dvc.yaml (v1) · 1493de1 params (v2).

A3. Log variants (captured when dev was one commit ahead of main)

```
~/Desktop/Code/fashion-ann-pipeline % git log --oneline --graph --all
* @9ac6bd (HEAD -> dev, origin/dev) feat: added data download script
* @6ee428 (origin/main, origin/HEAD, main) init: fashion-ann-pipeline project with essential files
~/Desktop/Code/fashion-ann-pipeline %
```

--oneline --graph --all: one line per commit across all branches and remotes, with the topology drawn (plain git log shows only the current branch, verbosely).

```
~/Desktop/Code/fashion-ann-pipeline % git log --stat -3
commit @9ac6bd1ce2b3dd7786db1b18d118ee4141f46 (HEAD -> dev, origin/dev)
Author: Samiha Hamid <s.mia131103@gmail.com>
Date: Sat Oct 3 01:04:22 2026 +0500

feat: added data download script

- Added data directory to .gitignore to exclude raw data files.
- Updated pyproject.toml to include TensorFlow as a dependency.
- Introduced src/prepare.py for downloading and saving Fashion-MNIST dataset.
- Removed unused __init__.py from fashion_ann_pipeline module.

.gitignore      | 3 +
pyproject.toml  | 11 +-
src/fashion_ann_pipeline/__init__.py | 2 -
src/prepare.py  | 27 +++
uv.lock         | 637 +++++
5 files changed, 669 insertions(+), 11 deletions(-)
```

--stat -3 (cropped): files touched by each of the last 3 commits, with insertion/deletion counts.

```
~/Desktop/Code/fashion-ann-pipeline % git log -p -1
commit @9ac6bd1ce2b3dd7786db1b18d118ee4141f46 (HEAD -> dev, origin/dev)
Author: Samiha Hamid <s.mia131103@gmail.com>
Date: Sat Oct 3 01:04:22 2026 +0500

feat: added data download script

- Added data directory to .gitignore to exclude raw data files.
- Updated pyproject.toml to include TensorFlow as a dependency.
- Introduced src/prepare.py for downloading and saving Fashion-MNIST dataset.
- Removed unused __init__.py from fashion_ann_pipeline module.

diff --git a/.gitignore b/.gitignore
index 505a3b1..cb521fd 100644
```

-p -1 (cropped): the full patch of the latest commit, i.e. what changed line by line.

```
~/Desktop/Code/fashion-ann-pipeline % git log main..dev
commit @9ac6bd1ce2b3dd7786db1b18d118ee4141f46 (HEAD -> dev, origin/dev)
Author: Samiha Hamid <s.mia131103@gmail.com>
Date: Sat Oct 3 01:04:22 2026 +0500

feat: added data download script

- Added data directory to .gitignore to exclude raw data files.
- Updated pyproject.toml to include TensorFlow as a dependency.
- Introduced src/prepare.py for downloading and saving Fashion-MNIST dataset.
- Removed unused __init__.py from fashion_ann_pipeline module.
~/Desktop/Code/fashion-ann-pipeline %
```

main..dev: only commits on dev that are not on main, i.e. what merging dev would bring in.

A4. Diff variants

```
~/Desktop/Code/fashion-ann-pipeline % git diff
diff --git a/pyproject.toml b/pyproject.toml
index bf272cc..9a6d40e 100644
--- a/pyproject.toml
+++ b/pyproject.toml
@@ -8,5 +8,6 @@ authors = [
 ]
 requires-python = ">=3.14"
 dependencies = [
+    "pyyaml==6.0.3",
     "tensorflow==2.22.0rc0",
 ]
```

(a) git diff: working tree vs index (unstaged pyyaml addition)

```
~/Desktop/Code/fashion-ann-pipeline % git diff --staged
diff --git a/.gitignore b/.gitignore
index cb521fd..a6b6a18 100644
--- a/.gitignore
+++ b/.gitignore
@@ -11,5 +11,6 @@ wheels/

# Data
data/
+
+# Submission files
+screenshots/
~/Desktop/Code/fashion-ann-pipeline %
```

(b) git diff --staged: index vs HEAD (staged .gitignore line)

```
~/Desktop/Code/fashion-ann-pipeline % git diff main..dev
diff --git a/.gitignore b/.gitignore
index 505a3b1..cb521fd 100644
--- a/.gitignore
+++ b/.gitignore
@@ -8,3 +8,6 @@ wheels/

# Virtual environments
venv/
```

(c) main..dev (cropped)

```
~/Desktop/Code/fashion-ann-pipeline % git diff main..dev
diff --git a/.gitignore b/.gitignore
index 505a3b1..cb521fd 100644
--- a/.gitignore
+++ b/.gitignore
@@ -8,3 +8,6 @@ wheels/

# Virtual environments
venv/
```

(c) main..dev (cropped)

Two-dot vs three-dot. main..dev compares the two tips, so commits made on main after branching show up too, reversed. main...dev compares the merge-base with dev: only what dev introduced. The screenshots match because main had not moved yet; the Part E branches show the difference:

```
$ git diff --stat b704885..teammate-sim      # tip vs tip
src/preprocess.py | 26 ++++++-----         <- includes main's own "/" 127.5 - 1.0" change, reversed
$ git diff --stat b704885...teammate-sim     # merge-base vs tip
src/preprocess.py | 22 ++++++-----         <- only teammate-sim's scale()/standardize()
```

A5. Stash

```
~/Desktop/Code/fashion-ann-pipeline % git stash
Saved working directory and index state WIP on dev: @9ac6bd feat: added data download script
~/Desktop/Code/fashion-ann-pipeline % git checkout main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
~/Desktop/Code/fashion-ann-pipeline % git checkout dev
Switched to branch 'dev'
Your branch is up to date with 'origin/dev'.
~/Desktop/Code/fashion-ann-pipeline % git stash list
stash@0: WIP on dev: @9ac6bd feat: added data download script
~/Desktop/Code/fashion-ann-pipeline % git stash pop
On branch dev
Your branch is up to date with 'origin/dev'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   .gitignore
        modified:   pyproject.toml
        modified:   uv.lock

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        params.yaml
        src/preprocess.py

no changes added to commit (use "git add" and/or "git commit -a")
Dropped refs/stash@0 (d6e64e437457e3b720b56c14e962e9a5181ceb)
~/Desktop/Code/fashion-ann-pipeline %
```

Here preprocess.py was still *untracked*, so plain git stash left it in the working tree (-u would include it). Repeated once it was tracked:

```
$ git stash      # with src/preprocess.py modified
Saved working directory and index state WIP on dev: 1493de1
$ git switch main && git switch dev
$ git stash list
stash@{0}: WIP on dev: 1493de1 feat: enhance model ...
$ git stash pop
        modified:   src/preprocess.py
```

Original run

A6. Rebase onto a hotfix

```
~/Desktop/Code/fashion-ann-pipeline % git log --oneline --graph --all
* b6e1cbb (HEAD -> main, origin/main, origin/HEAD, hotfix) docs: add project description to README.md for fashion-ann-pipeline
* b430450 (origin/dev, dev) feat: added preprocessing and dependencies
* 09acb6d feat: added data download script
* 6fec428 init: fashion-ann-pipeline project with essential files
~/Desktop/Code/fashion-ann-pipeline %
```

Before

```
~/Desktop/Code/fashion-ann-pipeline % git log --oneline --graph --all
* 6d0e33a (HEAD -> dev, origin/dev) feat: added preprocessing and dependencies
* 7f5d237 feat: added data download script
* b6e1cbb (origin/main, origin/HEAD, main, hotfix) docs: add project description to README.md for fashion-ann-pipeline
* 6fec428 init: fashion-ann-pipeline project with essential files
~/Desktop/Code/fashion-ann-pipeline %
```

After git rebase main

A7. Reset (soft vs hard)

```
~/Desktop/Code/fashion-ann-pipeline % git branch
dev
hotfix
main
~/Desktop/Code/fashion-ann-pipeline % git reset --soft HEAD~1
~/Desktop/Code/fashion-ann-pipeline % git status
On branch hotfix
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   README.md

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        data/
        screenshots/

~/Desktop/Code/fashion-ann-pipeline % git reset --hard HEAD~1
HEAD is now at b6e1cbb docs: add project description to README.md for fashion-ann-pipeline
~/Desktop/Code/fashion-ann-pipeline % git status
On branch hotfix
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        data/
        screenshots/

nothing added to commit but untracked files present (use "git add" to track)
~/Desktop/Code/fashion-ann-pipeline %
```

hotfix (from main) got one README fix, b6e1cbb, and was fast-forward merged into main. git rebase main on dev then replayed dev's commits on top as new commits (09acb6d→7f5d237, b430450→dfb8989): linear history, no merge commit. No conflict arose since dev never touched README.md (reflog: rebase (finish): refs/heads/dev onto b6e1cbb).

```
# git reflog (scratch branch = the already-merged hotfix)
6d0e33a commit: docs: added throwaway commit 1
01aeb7 commit: docs: added throwaway commit 2
6d0e33a reset: moving to HEAD~1  ← --soft
b6e1cbb reset: moving to HEAD~1  ← --hard
```

Observed: --soft moved only the branch pointer, so the undone edit stayed staged. --hard also reset the index and working tree: the edit was discarded and only untracked files remained.

A8. git mv and git rm

```
~/Desktop/Code/fashion-ann-pipeline % git mv train.py src/train.py
~/Desktop/Code/fashion-ann-pipeline % git rm params.yml
rm 'params.yml'
~/Desktop/Code/fashion-ann-pipeline % git commit -m "refactor: move train.py into src/ and remove yml typo file"
(dev 1127765) refactor: move train.py into src/ and remove yml typo file
2 files changed, 0 insertions(+), 0 deletions(-)
delete mode 100644 params.yml
rename train.py => src/train.py (100%)
~/Desktop/Code/fashion-ann-pipeline % git show --stat -M HEAD
commit 1127765a8bb15fe2e21b2ee1e6ed612914fb75e (HEAD -> dev)
Author: Samiha Hamid <s.mia1310@gmail.com>
Date: Sat Oct 3 02:18:07 2026 +0500

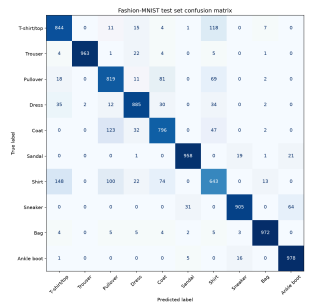
    refactor: move train.py into src/ and remove yml typo file

 params.yml | 0
 train.py => src/train.py | 0
 2 files changed, 0 insertions(+), 0 deletions(-)
```

git mv train.py src/train.py and git rm params.yml (a typo duplicate of params.yaml) in one commit, 1127765. Git records a rename ... (100%) and a delete mode, not a delete plus a new file, so git log --follow src/train.py keeps the history.

Part B: Modular TensorFlow ANN pipeline

Script	Reads → writes	What it does
prepare.py	Keras dataset → data/raw/{train,test}.npz	Saves the raw 60k / 10k uint8 28×28 images; no params.
preprocess.py	data/raw → data/processed/{train,val,test}.npz	Seeded shuffle, val_split 0.1 → 54k / 6k / 10k; scales to [0, 1] (v1, v2), and since Part E also standardizes.
train.py	processed/{train,val} → models/model.h5, history.csv	Flatten → Dense(hidden_units, ReLU) → Dropout(dropout_rate) → Dense(10, softmax); Adam(learning_rate), sparse categorical CE, seeded.
evaluate.py	model.h5 + processed/test → metrics.json, confusion_matrix.png	Test loss and accuracy, plus the confusion-matrix image.



models/confusion_matrix.png

Each script runs standalone (python src/<name>.py) and takes its hyperparameters from params.yaml (see D1). In the confusion matrix, Shirt is the weakest class (643/1000), mostly confused with T-shirt, Pullover and Coat.

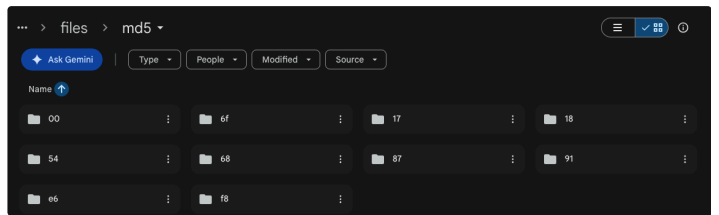
Part C: DVC with Google Drive as remote

- C1.** dvc + dvc-gdrive installed in the project venv (via uv; equivalent to pip install "dvc[gdrive]"). dvc doctor → DVC version: 3.67.1, Supports: gdrive (pydrive2 = 1.21.2). cryptography<44 was pinned for compatibility (c986e14).
- C2.** dvc init ran on dev after the Part A commits (c5f22d0: .dvc/config, .dvc/.gitignore, .dvcignore); re-initialised once on dev (a669c5d) when switching to a personal OAuth client.
- C3.** dvc remote add -d gdriveremote gdrive://1ngK-GvQZZmEW2jNMASRhXAomcZwGj07z (first named gdrive_storage, renamed in 35d1eb5).

- **C4.** Google blocks DVC's shared OAuth app, so a personal OAuth client is used. Its ID/secret are in the git-ignored `.dvc/config.local` (the committed `.dvc/config` has only ever held placeholders). The OAuth token is cached at `~/Library/Caches/pydrive2fs/<client-id>/default.json`, **outside the repository**. The ignore rules are verified below.
- **C5.** `dvc add data/raw data/processed models` → `data/raw.dvc`, `data/processed.dvc`, `models.dvc` and `data/.gitignore`, committed in 0e53484, then `dvc push`. When `dvc.yaml` arrived (7b71205) these were removed, since a path cannot be both a `dvc add` target and a stage output; the same hashes moved into `dvc.lock`.

```
# data/processed.dvc (commit 0e53484)
outs:
- md5: 005380317322710a7d01e06eed8e6cd0.dir
  size: 45391160
  nfiles: 3
  path: processed
```

```
$ git check-ignore -v .env .dvc/config.local .dvc/cache
models/model.h5 data/processed
.gitignore:14:.env                .env
.dvc/.gitignore:1:/config.local  .dvc/config.local
.dvc/.gitignore:3:/cache         .dvc/cache
.gitignore:20:/models           models/model.h5
data/.gitignore:2:/processed    data/processed
```



Drive remote after `dvc push`: DVC's content-addressed cache (files/md5/...)

Part D: DVC pipeline (params.yaml + dvc.yaml)

```
# params.yaml (final, verbatim)
preprocess:
  val_split: 0.1 # fraction of the training data held out for validation
  seed: 42       # RNG seed for the shuffle before splitting

train:
  hidden_units: 256 # neurons in the Dense (ReLU) layer
  dropout_rate: 0.2 # fraction of units dropped after the hidden layer
  learning_rate: 0.001 # Adam learning rate
  epochs: 10
  batch_size: 32
  seed: 42 # seed for Python / NumPy / TensorFlow RNGs
```

```
# dvc.yaml (final; lists shown in flow style,
# the committed file uses block style)
stages:
  prepare:
    cmd: python src/prepare.py
    deps: [src/prepare.py]
    outs: [data/raw]
  preprocess:
    cmd: python src/preprocess.py
    deps: [src/preprocess.py, data/raw]
    params: [preprocess.val_split, preprocess.seed]
    outs: [data/processed]
  train:
    cmd: python src/train.py
    deps: [src/train.py, data/processed/train.npz,
           data/processed/val.npz]
    params: [train.hidden_units, train.dropout_rate,
             train.learning_rate, train.epochs,
             train.batch_size, train.seed]
    outs: [models/model.h5, models/history.csv]
  evaluate:
    cmd: python src/evaluate.py
    deps: [src/evaluate.py, models/model.h5,
           data/processed/test.npz]
    outs: [models/confusion_matrix.png]
    metrics:
      - metrics.json:
        cache: false
```

D1. `preprocess.py` and `train.py` read their own section with `yaml.safe_load()` (optional `--params path`); nothing is hard-coded.

D2. Each stage lists its script and inputs as `deps` and exactly the `params` keys it reads. `metrics.json` is a metrics entry with `cache: false`, so it stays in Git where `dvc metrics diff` can compare versions.

```
~/Desktop/Code/fashion-ann-pipeline % dvc repro
Running stage 'prepare':
> python src/prepare.py
train (60000, 28, 28), test (10000, 28, 28) -> /Users/ayesh/Desktop/Code/fashion-ann-pipeline/data/raw
Generating lock file 'dvc.lock'
Updating lock file 'dvc.lock'

Running stage 'preprocess':
> python src/preprocess.py
train: images (54000, 28, 28) float32
val : images (6000, 28, 28) float32
test : images (10000, 28, 28) float32
Updating lock file 'dvc.lock'

Running stage 'train':
> python src/train.py
-----
Running stage 'evaluate':
> python src/evaluate.py
WARNING:absl:Compiled the loaded model, but the compiled metrics have yet to be built. 'model.compile_metrics' will be
empty until you train or evaluate the model.
313/313 - 0s - 490us/step - accuracy: 0.8774 - loss: 0.3467
{
  "test_loss": 0.3466505706310272,
  "test_accuracy": 0.8773999810218811,
  "num_test_samples": 10000
}
Updating lock file 'dvc.lock'
```

D3: first `dvc repro`; all four stages run and `dvc.lock` is generated (model summary and epochs cut at the dashed line).

```
~/Desktop/Code/fashion-ann-pipeline % dvc repro
Stage 'prepare' didn't change, skipping
Stage 'preprocess' didn't change, skipping
Running stage 'train':
> python src/train.py
Model: "sequential"

Layer (type)                Output Shape         Param #
-----
flatten (Flatten)           (None, 784)          0
dense (Dense)                (None, 256)          200,960
dropout (Dropout)           (None, 256)          0
dense_1 (Dense)              (None, 10)           2,570

Total params: 203,530 (795.04 KB)
Trainable params: 203,530 (795.04 KB)
Non-trainable params: 0 (0.00 B)

Running stage 'evaluate':
> python src/evaluate.py
WARNING:absl:Compiled the loaded model, but the compiled metrics have yet to be built. 'model.compile_metrics' will be
empty until you train or evaluate the model.
313/313 - 0s - 527us/step - accuracy: 0.8814 - loss: 0.3404
{
  "test_loss": 0.34042981266975483,
  "test_accuracy": 0.8813999891261128,
  "num_test_samples": 10000
}
Updating lock file 'dvc.lock'
```

D4: `dvc repro` after `hidden_units 128` → 256.

D4: what re-ran and why. `prepare` and `preprocess` were **skipped**; `train` and `evaluate` **re-ran**. DVC compares the hashes of each stage's `deps` and declared `params` with `dvc.lock`. Only `train` declares `train.hidden_units`, so only it was stale; its new `model.h5` is a dependency of `evaluate`, so that re-ran too. Nothing the first two stages depend on had changed.

D5. `params.yaml`, `dvc.lock` and `metrics.json` committed and tagged `v2 (1493de1)`; `dvc.yaml` was already in `v1`, then `dvc push`.

Version	train.hidden_units	Trainable params	Val. acc. (epoch 10)	Test accuracy	Test loss
v1 (7b71205)	128	101,770	0.8918	0.8774	0.3467
v2 (1493de1)	256	203,530	0.8922	0.8814	0.3404
Change	x2	+101,760	+0.0004	+0.0040	-0.0062

Values match `dvc metrics diff v1 v2`. Training is seeded: the first stand-alone run (1663628) and the D3 run both gave exactly 0.8773999810218811.

Part E: Simulated collaboration and conflict resolution

E1-E2. `teammate-sim` was branched from `main` after the v1/v2 merge (3fc23d2). Each side changed normalization, ran `dvc repro` and `dvc push`, and committed code, `dvc.lock` and `metrics.json`. Because `data/processed` is a pipeline output, its pointer (the `.dir` hash) lives in `dvc.lock`, which plays the role of the `.dvc` file.

Branch, commit	Normalization in <code>preprocess.py</code>	<code>data/processed</code> hash	Test acc.
<code>teammate-sim</code> 077db33 (E1)	Scale to [0, 1], then standardize with the mean/std of the training split only	<code>ed0357f...a15144.dir</code>	0.8763
<code>main</code> b704885 (E2)	Rescale to [-1, 1] ($x / 127.5 - 1$)	<code>5c875e3...a9853.dir</code>	0.8805

E3. `git merge teammate-sim` on `main`:

```

fashion-ann-pipeline % git checkout main && git merge teammate-sim
Already on 'main'
Your branch is up to date with 'origin/main'.
Auto-merging dvc.lock
CONFLICT (content): Merge conflict in dvc.lock
Auto-merging metrics.json
CONFLICT (content): Merge conflict in metrics.json
Auto-merging src/preprocess.py
CONFLICT (content): Merge conflict in src/preprocess.py
Automatic merge failed; fix conflicts and then commit the result.
fashion-ann-pipeline %

src > preprocess.py > ...
-----
38 <----- HEAD (Current Change)
39 def normalize(images: np.ndarray) -> np.ndarray:
40     """Scale uint8 pixel values from [0, 255] to float32 in [-1, 1]."""
41     return images.astype(np.float32) / 127.5 - 1.0
42
43 def scale(images: np.ndarray) -> np.ndarray:
44     """Scale uint8 pixel values from [0, 255] to float32 in [0, 1]."""
45     return images.astype(np.float32) / 255.0

```

Code conflict: `normalize()` vs `scale()` in `src/preprocess.py`

```

dvc.lock
Accept Current Change | Accept Incoming Change | Accept Both Changes | Compare Changes | Resolve with AI
26 <----- HEAD (Current Change)
27 md5: 2121bd98cb2ce4b1ce8a91db7b18c398
28 size: 2249
29
30 md5: cc5007c583b7485919a961244e4e803f
31 size: 2764
32 >----- teammate-sim (Incoming Change)
-----
Accept Current Change | Accept Incoming Change | Accept Both Changes | Compare Changes | Resolve with AI
40 <----- HEAD (Current Change)
41 md5: 5c875e3de931bfcc3b8efb2b698a9853.dir
42 size: 47449678
43
44 md5: ed0357f9aa084e57874ed98660a15144.dir
45 size: 44962591
46 >----- teammate-sim (Incoming Change)

```

Data conflict in `dvc.lock`: the `preprocess.py` hash and the two `data/processed` hashes

E4: resolution. *Code:* both are affine rescalings. `main`'s only re-centres the range; `teammate-sim`'s yields zero-mean, unit-variance inputs from training-split statistics only, so `val/test` never leak into preprocessing. I kept `scale()` + `standardize()` and dropped `normalize()`. The 0.4-point accuracy gap is one seeded run per side, so it isn't strong evidence either way. *Data:* `teammate-sim`'s `data/processed` (`ed0357f...`) is authoritative because it is the output of the kept code, so I took that side's `dvc.lock` and `metrics.json` hunks. `dvc checkout` then syncs the workspace to the resolved pointer.

E5. Merge committed as 9d5d291 (parents: E2 b704885, E1 077db33) and pushed to GitHub (the chosen data was already on Drive from E1). Re-checked on the final merged state:

```

$ git log --oneline --graph -5
* 9d5d291 (HEAD -> main, origin/main, origin/HEAD) Merge branch 'teammate-sim'
|
| * 077db33 (origin/teammate-sim, teammate-sim) fix: update preprocessing ...
* | b704885 fix: update preprocessing normalization and adjust DVC outputs
|/
* 3fc23d2 Merge branch 'dev'
$ dvc status -c --all-branches --all-tags
Cache and remote 'gdriverremote' are in sync.

```

```

$ dvc checkout && dvc status
Data and pipelines are up to date.
$ dvc repro
Stage 'prepare' didn't change, skipping
Stage 'preprocess' didn't change, skipping
Stage 'train' didn't change, skipping
Stage 'evaluate' didn't change, skipping
Data and pipelines are up to date.

```

Every stage's hashes match the committed `dvc.lock`, so the merged state reproduces as committed.